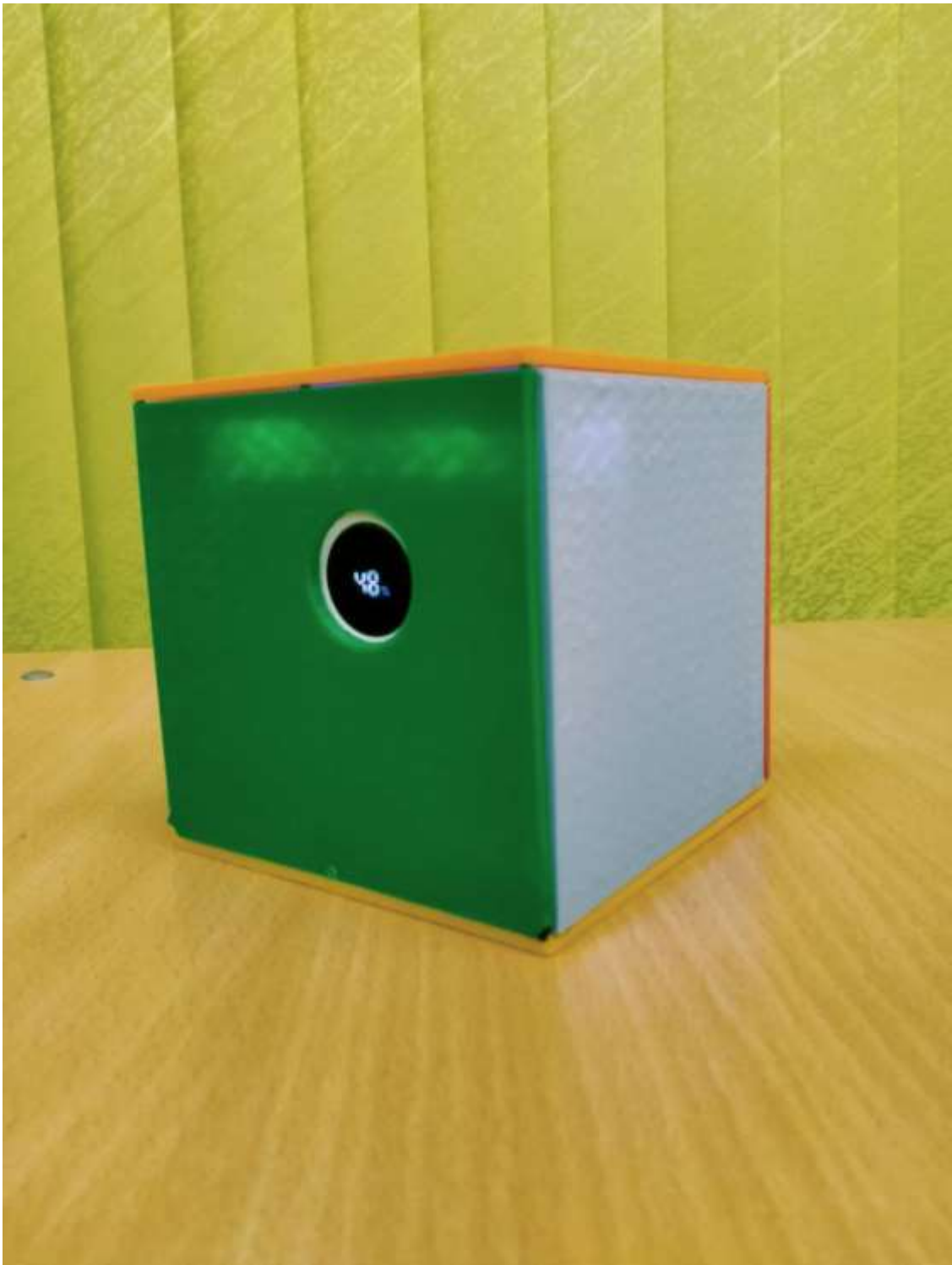


# **ИНТЕЛЛЕКТУАЛЬНЫЕ КУБЫ, СИНХРОНИЗИРУЮЩИЕ СТАТУСЫ РАЗЛИЧНОГО СОСТОЯНИЯ УЧЕНИКОВ**



*Рисунок 1 Фотография проекта*

## РЕФЕРАТ

- **Пояснительная записка** изложена на 40 страницах, содержит 13 рисунков, 3 таблицы, 7 источников, 2 приложения.
- **Объект разработки:** Система интеллектуальных кубов для тактильной передачи статусов учеников в классе. Устройство (110×110×110 мм) включает ESP-12F, MPU6050 и аккумулятор.
- **Цель работы:** Создание рабочего прототипа для бесшумного обмена статусами между учениками и учителем в реальном времени.
- **Методы исследования, проектирования, дизайнерской работы:** Анализ аналогов, 3D-моделирование, схемотехника, программирование (Arduino/C++, MQTT), опытное проектирование.
- **Результаты работы и их новизна:** Разработан функциональный прототип. Новизна — оригинальная технология групповой калибровки, тактильный интерфейс и ПО для аналитики.
- **Область применения результатов:** Средние общеобразовательные школы, инклюзивные и корпоративные среды.
- **Рекомендации по внедрению:** Пилотное тестирование в 1-2 классах, интеграция с WiFi.
- **Экономическая эффективность:** Себестоимость прототипа — 1750 руб., рыночная цена — 2750 руб. Рентабельность серийного производства — свыше 55%
- **Перспективы:** Масштабирование на корпоративный сектор, разработка облачной аналитической платформы, переход на литьё под давлением для массового производства.
- **Ключевые слова:** Синхронизация статусов, интеллектуальный куб, беспроводная связь WiFi, 3D-печать, образовательные технологии, децентрализованная коммуникация, IoT-устройство, корпусная электроника, прототипирование.

## СОДЕРЖАНИЕ

|                                                                                              |           |
|----------------------------------------------------------------------------------------------|-----------|
| <b>ВВЕДЕНИЕ .....</b>                                                                        | <b>6</b>  |
| <b>1 ПОИСКОВО-ИССЛЕДОВАТЕЛЬСКИЙ ЭТАП .....</b>                                               | <b>8</b>  |
| 1.1 Актуальность темы проекта .....                                                          | 8         |
| 1.2 Цели и задачи проекта .....                                                              | 8         |
| 1.3 Сбор информации по проблеме, решаемой в проекте .....                                    | 9         |
| 1.3.1 Результаты маркетингового исследования .....                                           | 9         |
| 1.3.2 Характеристики проектируемого объекта труда, для потенциальной целевой аудитории ..... | 9         |
| 1.4 Анализ прототипов и аналогов .....                                                       | 10        |
| <b>2 КОНСТРУКТОРСКО-ТЕХНОЛОГИЧЕСКИЙ ЭТАП .....</b>                                           | <b>11</b> |
| 2.1 Разработка идеи и концепции проекта .....                                                | 11        |
| 2.2 Применение методов проектирования и дизайнерской работы .....                            | 11        |
| 2.2.1 Применение методов проектирования при разработке конструкции .....                     | 11        |
| 2.2.4 Применение АРИЗ .....                                                                  | 13        |
| 2.3 Выбор технологии изготовления проектируемого изделия .....                               | 14        |
| 2.4 Описание принципа действия и конструкции .....                                           | 16        |
| 2.4.1 Принцип действия .....                                                                 | 16        |
| 2.4.2 Общая конструкция .....                                                                | 17        |
| 2.5 Графическая документация и 3D моделирование .....                                        | 18        |
| 2.5.1 Изображение сборки проекта .....                                                       | 18        |
| 2.5.2 Техническая карта проекта .....                                                        | 19        |
| <b>3 ЗАКЛЮЧИТЕЛЬНЫЙ ЭТАП .....</b>                                                           | <b>20</b> |

|                                                          |           |
|----------------------------------------------------------|-----------|
| 3.1 Эколого-экономическое обоснование .....              | 20        |
| 3.1.1 Экономический расчёт .....                         | 20        |
| 3.1.2 Экологическая оценка .....                         | 20        |
| 3.2 Анализ и обобщение результатов проектной работы..... | 22        |
| 3.3 Предложения по дальнейшему развитию проекта .....    | 23        |
| <b>4 СПИСОК ИСТОЧНИКОВ .....</b>                         | <b>25</b> |
| Приложение А.....                                        | 26        |
| Приложение В.....                                        | 29        |

## ВВЕДЕНИЕ

Современный образовательный процесс сталкивается с классической дилеммой: как эффективно организовать оперативную обратную связь между учителем и всем классом, не нарушая динамику урока и не создавая излишнего шума. Традиционный метод поднятия руки или использование бумажных карточек имеет существенные ограничения — он не позволяет мгновенно оценить общую картину понимания материала, требует постоянного визуального контроля со стороны педагога и не предоставляет возможности для сбора и анализа данных об учебной активности.

В ответ на этот вызов был разработан проект «Интеллектуальные кубы, синхронизирующие статусы различного состояния учеников». Данное устройство представляет собой инновационное решение, сочетающее интуитивный тактильный интерфейс с современными IoT-технологиями. Каждый куб, оснащённый датчиком пространственной ориентации и WiFi-модулем, позволяет ученику бесшумно сообщать о своём состоянии (например, «Готов», «Нужна помощь», «Выполнил задание») простым поворотом нужной грани вверх. Данные со всех устройств в классе в реальном времени передаются на компьютер педагога, где визуализируются в виде наглядной схемы.

Целью проекта является не просто создание ещё одного гаджета, а разработка целостной системы тактильной коммуникации, которая решает ключевые проблемы существующих аналогов: высокую стоимость цифровых пультов, пассивность механических карточек и узкую специализацию инклюзивных устройств. Проект предлагает универсальный, экономичный и масштабируемый подход, способный повысить эффективность учебного процесса, снизить организационную нагрузку на учителя и создать комфортную, инклюзивную среду для каждого ученика.

В данной работе представлены результаты полного цикла проектирования — от анализа существующих решений и обоснования

концепции до разработки конструкции, программного обеспечения, экономического расчёта и оценки экологического воздействия.

# **1 ПОИСКОВО-ИССЛЕДОВАТЕЛЬСКИЙ ЭТАП**

## **1.1 Актуальность темы проекта**

Актуальность проекта обусловлена переходом на гибридное обучение, где эффективная, но ненавязчивая коммуникация критически важна. Традиционные методы — устные вопросы и цифровые сообщения — создают коммуникационный шум, который мгновенно прерывает глубокую концентрацию. Восстановление фокуса может занимать более 20 минут, что приводит к большим потерям продуктивного времени в группе. Проблема, которую решает проект "Интеллектуальные кубы", — в отсутствии инструмента для быстрой и тихой синхронизации статусов. Существующие способы либо нарушают работу других (вопросы вслух), либо требуют отвлечения на экран (мессенджеры), создавая новые помехи вместо их устранения. Для стеснительных учеников или в больших группах эта проблема обострена. Таким образом, существует потребность в решении, которое позволяло бы обмениваться статусами в реальном времени, оставаясь тактильным, бесшумным и не требующим взаимодействия с экраном. Проект Интеллектуальные кубы предлагает именно такой подход, переводя цифровую коммуникацию в физический жест.

## **1.2 Цели и задачи проекта**

Цель проекта: разработка и создание рабочего прототипа системы тактильной коммуникации на основе множества интеллектуальных кубов для бесшумного и мгновенного обмена статусами в учебных и рабочих пространствах.

Задачи проекта:

Провести исследование существующих проблем коммуникации в учебных аудиториях и офисах, а также проанализировать аналогичные решения для выявления их недостатков и определения ключевых требований к разрабатываемому продукту. Спроектировать, смоделировать и изготовить методом 3D-печати эргономичный и функциональный корпус устройства,



обеспечивающий удобство использования, надежное размещение электронных компонентов. Разработать, собрать и отладить электронную схему устройства, интегрирующую микроконтроллер (ESP-12F), датчик положения (MPU-6050) для реализации основной функциональности. Написать, загрузить и протестировать прошивку для микроконтроллера, которая обеспечивает:

- Корректное определение пространственной ориентации куба.
- Надежное сетевое взаимодействие для синхронизации данных между всеми устройствами в сети.
- Провести комплексное тестирование готового прототипа в условиях, приближенных к реальным, для оценки его работоспособности, надежности и удобства использования, а также сформулировать рекомендации по его дальнейшему внедрению и развитию.

### **1.3 Сбор информации по проблеме, решаемой в проекте**

#### **1.3.1 Результаты маркетингового исследования**

Преподаватели ищут инструмент для бесшумной обратной связи от всего класса, чтобы видеть, кому нужна помощь, не прерывая урок и не заставляя стеснительных учеников говорить публично, или для того, чтобы понять преподавателю, что у ребенка на уме и понимает ли он тему в данный момент. Традиционные методы — устный опрос или наблюдение — не дают полной и оперативной картины по всему классу одновременно, особенно если ученик стесняется или не хочет привлекать внимание.

#### **1.3.2 Характеристики проектируемого объекта труда, для потенциальной целевой аудитории**

- Анонимность и безопасность: Возможность честно показать, что тема не понята, не боясь осуждения со стороны сверстников или преподавателя.

- Интуитивность и скорость: Изменение статуса простым физическим поворотом куба, что не отвлекает от основной задачи и не требует цифровых навыков.
- Физический контроль: в рамках учебного процесса — взаимодействие с физическим объектом вместо очередного приложения или чата.

## 1.4 Анализ прототипов и аналогов

Таблица 1 Аналоги

| Категория аналога                                                          | Примеры                                        | Ключевые достоинства                          | Ключевые недостатки (подтверждающие актуальность проекта) | Требование к проекту (намерение устранить недостаток)                     |
|----------------------------------------------------------------------------|------------------------------------------------|-----------------------------------------------|-----------------------------------------------------------|---------------------------------------------------------------------------|
| Механические устройства для обратной связи в классе                        | Двусторонние карточки, таблички "Готов/Вопрос" | Дёшево, просто, надёжно                       | Видно только рядом, нет данных, мало статусов             | Сохранить простоту, добавить передачу данных и 6+ статусов                |
| Цифровые планшеты или специализированные пульты для голосования            | iClicker, планшеты с Kahoot!                   | Автоматический сбор данных, анализ            | Дорого, отвлекает на экран, сложный интерфейс             | Сделать дешёвый специализированный гаджет без экрана                      |
| Специализированные коммуникативные устройства для инклюзивного образования | BIGmack, GoTalk для детей с ОВЗ                | Адаптированы для особых потребностей, прочные | Очень дорогостоящее (15-100 тыс. Р)                       | Создать универсальное устройство для ВСЕХ по доступной цене (~1.5 тыс. Р) |

## **2 КОНСТРУКТОРСКО-ТЕХНОЛОГИЧЕСКИЙ ЭТАП**

### **2.1 Разработка идеи и концепции проекта**

Идея проекта:

Решить проблему коммуникационных проблем в группах, в классе с помощью не цифрового уведомления, а физического взаимодействия. Создать простой, интуитивно понятный объект, который позволит человеку молча, за секунду, показать своё состояние — нужна ли помощь, можно ли отвлекать, всё ли понятно.

Концепция проекта:

Реализовать идею через специально разработанный куб, выполняющий роль тактильного интерфейса для цифровой системы. Каждая из шести граней закреплена за конкретным статусом. Пользователь, меняя положение куба на столе, меняет свой статус в сети. Встроенный датчик положения считывает эту ориентацию, микроконтроллер обрабатывает сигнал и через WiFi транслирует его на центральный экран (для преподавателя). Таким образом, коммуникация становится ненавязчивой, мгновенной и бесшумной, не требуя ни голоса, ни взгляда на другой экран.

### **2.2 Применение методов проектирования и дизайнерской работы**

#### **2.2.1 Применение методов проектирования при разработке конструкции**

Для разработки конструкции использовался строгий последовательный инженерный подход. Это позволило методично пройти путь от замысла до готового прототипа, где каждый этап вытекал из предыдущего и служил основой для следующего. Системно-функциональный подход: Работа началась не с формы, а с функционального ядра. Были определены и подобраны ключевые компоненты, обеспечивающие требуемые функции: микроконтроллер ESP-12F (функция "мозг" и сетевая связь) и датчик MPU-6050 (функция "определение положения"). Это заложило техническую основу всей конструкции. Схемотехническое проектирование как основа: перед

любым 3D-моделированием была продумана и смоделирована электрическая принципиальная схема. Этот метод позволил виртуально проверить совместимость компонентов, правильность подключения и избежать критических ошибок на этапе физической сборки, обеспечив надёжность будущего устройства. Метод прототипирования "от простого к сложному": После проверки схемы был создан упрощённый 3D-макет корпуса для проверки габаритов и компоновки. Убедившись в правильности заложенных размеров, модель была доработана с учётом эргономики и технологичности печати, после чего отправлена на изготовление.

### **2.2.2 Применение методов дизайнерской работы при создании образа изделия**

Дизайн-работа велась параллельно с инженерной, фокусируясь на финальном восприятии изделия пользователем. Метод итеративной доводки формы: Цикл «3D-печать – физическая обработка (шлифовка, устранение дефектов) – оценка в руках» является ключевым методом тактильного дизайна. Обработка напечатанного корпуса позволила перевести виртуальную модель в реальный объект, оценить эргономику, вес, баланс и качество поверхностей, окончательно сформировав завершённый образ продукта. Принцип «Форма следует за функцией» (Form follows function): Внешний вид куба напрямую вытекает из его работы. Простая геометрическая форма (куб) была выбрана как наиболее очевидная и естественная для тактильного управления положением в пространстве, а также оптимальная для 3D-печати.

### **2.2.3 Применение методов при выборе материалов для изготовления**

Выбор материала был результатом анализа, а не случайностью.

**Критериальный анализ и тестирование:** Основным материалом для печати корпуса был выбран PLA-пластик. Решение принималось на основе оценки ключевых критериев: безопасность (не токсичность, что важно для учебного класса), технологичность (простота печати, минимальная усадка), доступность и достаточная прочность для устройства. Последующая обработка напечатанной детали оказалась удобной.

### **2.2.4 Применение АРИЗ**

Для решения ключевой задачи — как ученику передать статус без стресса и отвлечения — был применён АРИЗ.

Выявлено противоречие: Устройство должно быть интерактивным (принимать команду) и неинтерактивным (не отвлекать).

Сформулирован ИКР: Статус передаётся автоматически, без специальных действий ученика.

Найден ресурс: Использовано естественное действие ученика — тактильное упорядочивание предметов на столе (поворот куба).

Применён приём “Вынесения”: Вредную функцию (сознательный ввод) вынесли за пределы системы, встроив её в интуитивный жест.

Получено решение: Куб стал аналоговым преобразователем: его ориентация в гравитационном поле, изменяемая учеником, считывается датчиком и автоматически конвертируется в цифровой статус для учителя.

АРИЗ позволил создать не просто устройство, а новый принцип коммуникации — осязаемый интерфейс, исключающий цифровое отвлечение для ученика.

## 2.3 Выбор технологии изготовления проектируемого изделия

Выбор технологии и компонентов основывался на системном подходе, где каждый элемент определялся исходя из его функции в общей схеме и целевых требований проекта:

Первичным был выбор микроконтроллера ESP-12F как компонента, выполняющего ключевые функции “мозга” и сетевого хаба.

Основная задача — определить положение куба. Сравнивались варианты: механические кнопки (ненадёжны, много проводов), оптические датчики (сложны), магнитные датчики (требуют магнита). Выбран инерционный модуль MPU-6050.

Проблема в недостатке электроэнергии для работы всех компонентов решается с выбором надежного с большим ее запасом, аккумулятором PD20W.

Требование тактильности и доступность для школьной мастерской определили выбор аддитивной технологии (3D-печать) на принтере 3DQ. Материал PLA выбран за безопасность (нетоксичен при печати), простоту обработки и низкую усадку, что критично для точности посадки электроники.

Для пост-обработки использовалась наждачная бумага — обеспечения безопасности и эстетичного вида изделия.

Паяльная станция — пайка электронных компонентов, создание надежных соединений в схеме.

Для того, чтобы проверять на работоспособность соединение электрических цепей понадобился мультиметр.

В таблице 2 представлена таблица оборудования, инструментов и материалов

Таблица 2 Оборудования, инструменты и материалы

| № | Название компонента и его назначение в проекте                                                                                                                                               | Внешний вид                                                                           |
|---|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------|
| 1 | <p><b>ESP-12F(микроконтроллер)</b></p> <p>Наличие встроенного WiFi для автономной работы в сети, низкая цена, большое сообщество и обширные библиотеки для прототипирования.</p>             |    |
| 2 | <p><b>MPU-6050(датчик-положения)</b></p> <p>Измеряет гравитацию (акселерометр) для определения верхней грани, потребляет мало энергии и общается по простому интерфейсу (I2C) с ESP-12F.</p> |    |
| 3 | <p><b>Аккумулятор PD20W</b></p> <p>Используется для питания куба, обеспечения мобильности и длительной работы.</p>                                                                           |   |
| 4 | <p><b>3D-принтер 3DQ</b></p> <p>Используется для изготовления корпусов кубов методом послойного наплавления (FDM) из PLA-пластика по созданной 3D-модели.</p>                                |  |
| 5 | <p><b>PLA-пластик</b></p> <p>Биоразлагаемый и безопасный полимер для 3D-печати, из которого изготавливается прочный, лёгкий и эргономичный корпус куба.</p>                                  |  |

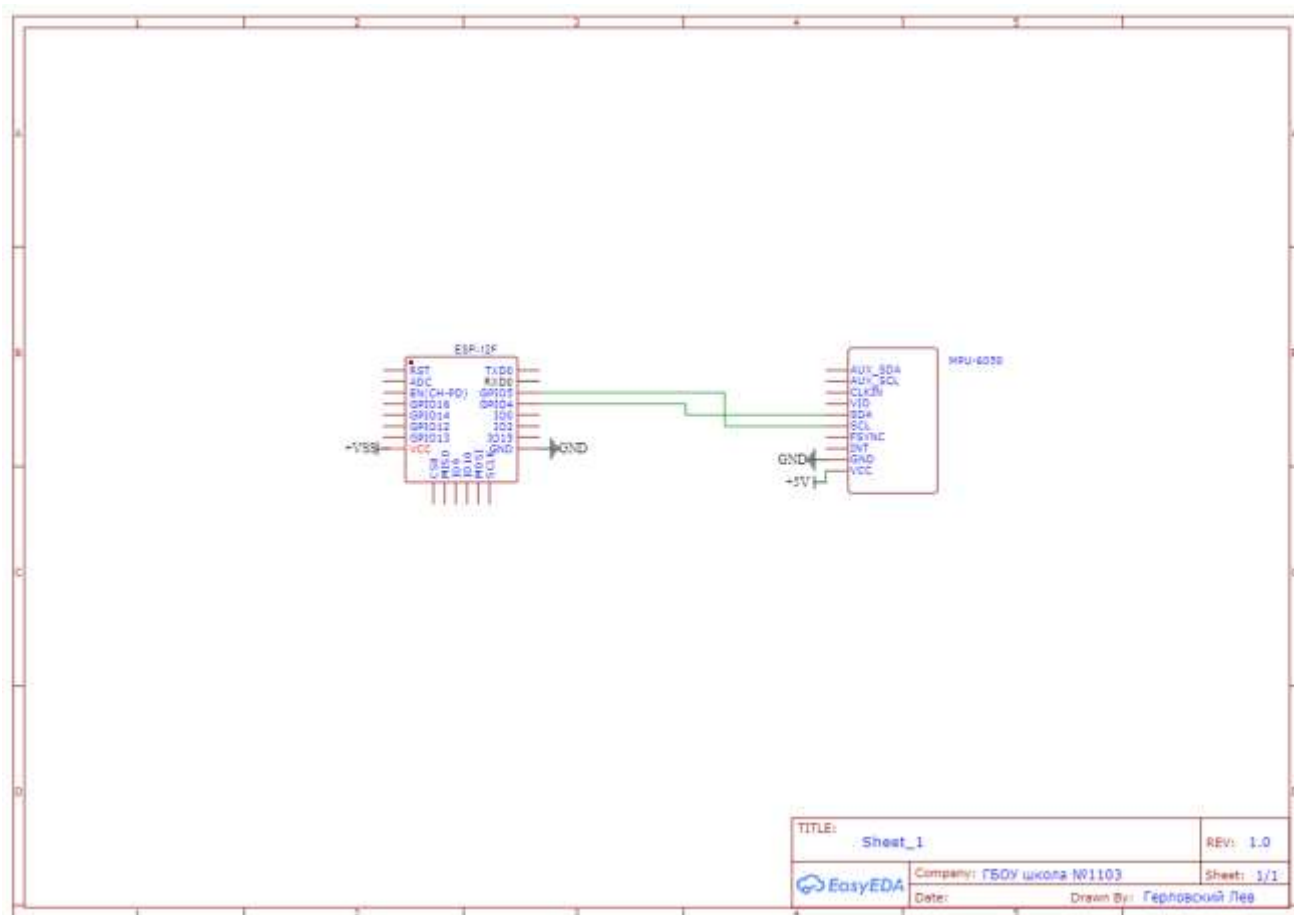


Рисунок 2 Электрическая схема

## 2.4 Описание принципа действия и конструкции

### 2.4.1 Принцип действия

Устройство работает на основе определения пространственной ориентации. Внутри каждого куба установлен инерциальный датчик MPU6050, который постоянно измеряет углы наклона относительно вектора гравитации. Когда ученик физически поворачивает куб другой гранью вверх, датчик фиксирует изменение ориентации. Микроконтроллер ESP-12F обрабатывает эти данные, преобразуя сырые показания акселерометра и гироскопа в конкретный номер активной грани (от 1 до 6). Каждой грани заранее присвоен определённый смысловой статус, например, «готов к работе», «нужна помощь» или «задание выполнено».



После определения статуса микроконтроллер через встроенный Wi-Fi-модуль устанавливает соединение с локальной сетью и отправляет данные на главный компьютер, используя лёгкий протокол MQTT. В передаваемом пакете содержится уникальный идентификатор куба (привязанный к ученику), текущий выбранный статус и служебная информация, например, уровень заряда батареи. Передача происходит в момент изменения статуса, а также периодически для подтверждения связи.

На стороне учителя специальное программное обеспечение, работающее на основном компьютере, получает данные от всех кубов в классе в реальном времени. Программа визуализирует информацию в виде интуитивной схемы рассадки, где каждый ученик представлен цветным индикатором, соответствующим его текущему статусу. Это позволяет педагогу мгновенно, без слов и поднятия рук, оценить общую картину понимания материала, готовность класса к следующему этапу урока или выявить учащихся, которым требуется индивидуальная помощь.

Для обеспечения обратной связи ученику, куб имеет определенный цвет, на каждой грани он разный. Каждому цвету присвоен определенный статус. Дополнительно на одной из граней может располагаться небольшой дисплей для отображения технической информации, например, оставшегося заряда батареи на аккумуляторе.

#### **2.4.2 Общая конструкция**

Конструкция устройства представляет собой сборочный куб с ребром 110 миллиметров, изготовленный методом аддитивного производства (3D-печати) из PLA-пластика. Данный материал обеспечивает оптимальный баланс между прочностью, весом и безопасностью для использования в учебных помещениях.

Внутреннее пространство куба разделено на функциональные зоны. На нижней грани расположена основная электронная плата. На ней смонтирован

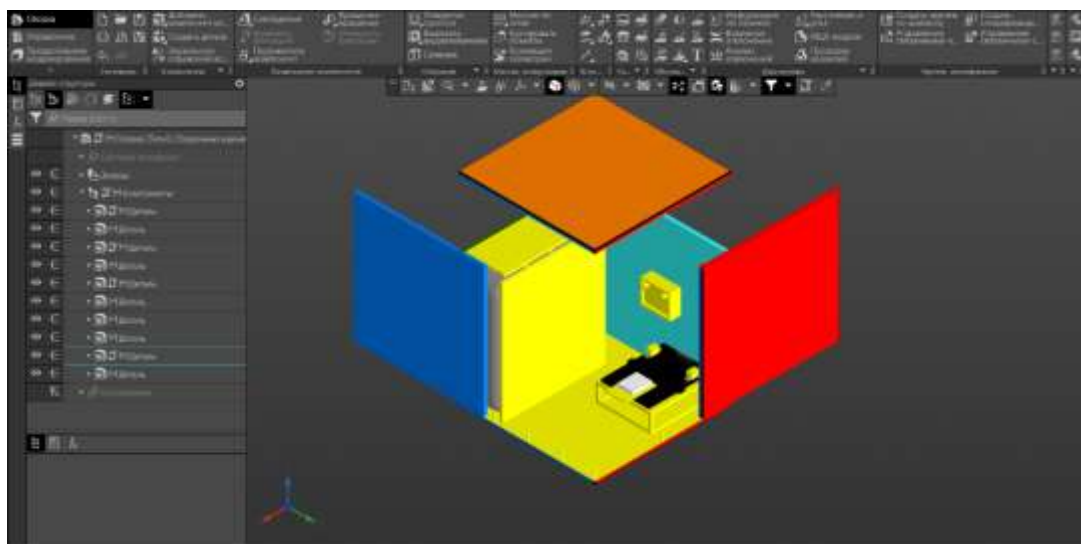
микроконтроллерный модуль ESP-12F и датчик ориентации MPU6050, жёстко зафиксированный и откалиброванный относительно осей корпуса.

В нижней части корпуса размещён аккумуляторный отсек, где находится прямоугольный, с закругленными краями, пластиковый аккумулятор ёмкостью 20000 миллиампер в час. Отсек оборудован съёмной крышкой для доступа к батарее. Включение аккумулятора производится по нажатию грани в области его расположения. Внешне устройство выглядит как сборочный предмет без видимых кнопок или разъёмов, что подчёркивает простоту взаимодействия.

## 2.5 Графическая документация и 3D моделирование

Все детали корпуса и электронные посадочные места были смоделированы в системе автоматизированного проектирования КОМПАС-3D v23. Для электрических схем использовалась программа Easy EDA. Ниже представлена 3D-сборка устройства и тех-карта основной модели:

### 2.5.1 Изображение сборки проекта



*Рисунок 3 Сборка*

2.5.2 Техническая карта проекта

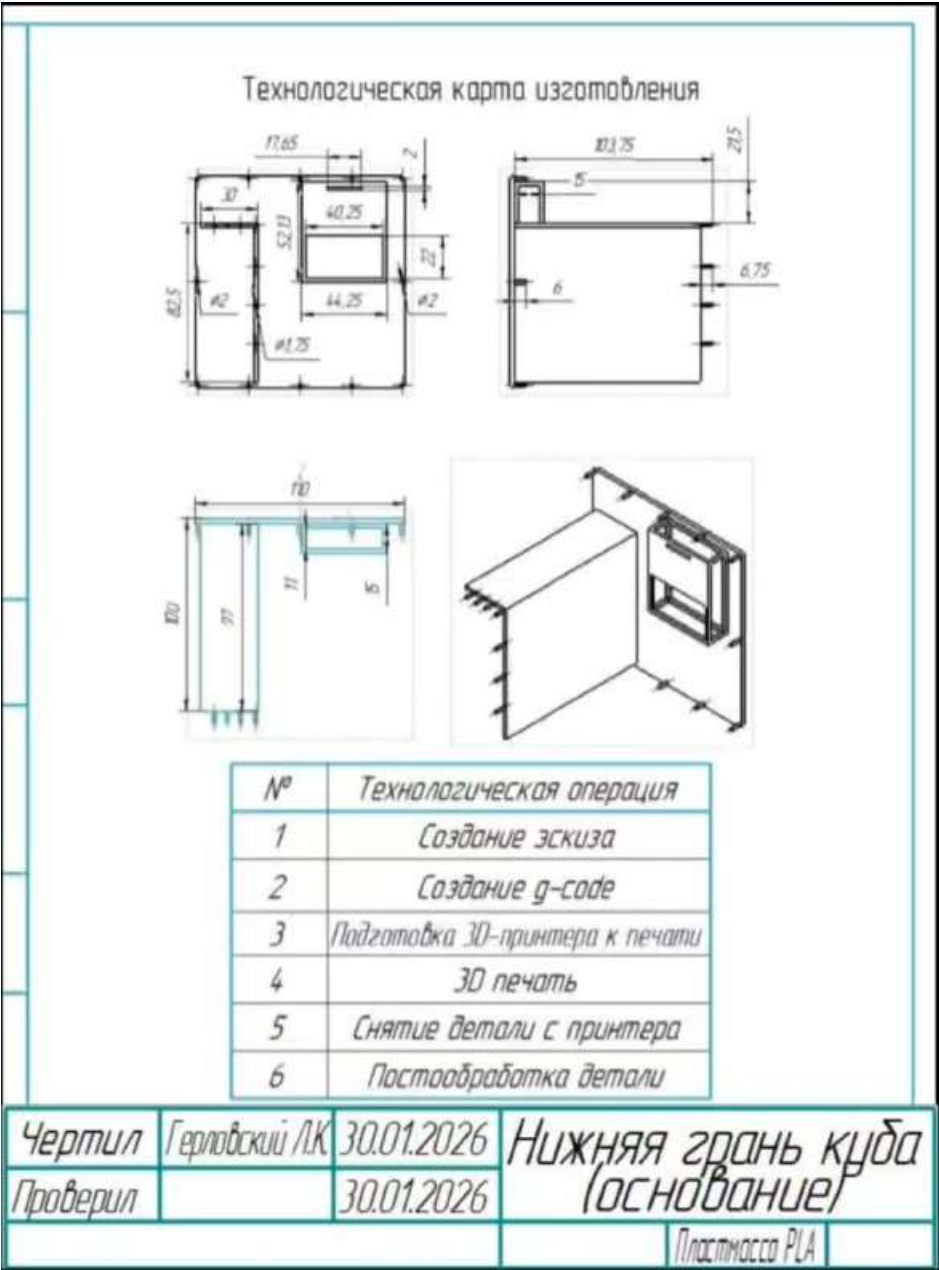


Рисунок 4 Тех-карта

Остальные 3D-модели – в Приложении.

### 3 ЗАКЛЮЧИТЕЛЬНЫЙ ЭТАП

#### 3.1 Эколого-экономическое обоснование

##### 3.1.1 Экономический расчёт

*Таблица 3 Экономический расчёт*

| Наименование                | Кол-во | Цена за ед., руб. | Сумма, руб. |
|-----------------------------|--------|-------------------|-------------|
| ESP-12F                     | 10     | 200               | 2000        |
| MPU-6050                    | 10     | 150               | 1500        |
| Аккумулятор<br>PD20W        | 10     | 1000              | 10000       |
| PLA-пластик                 | 3 кг   | 1250/кг           | 3750        |
| Крепёж, провода,<br>разъёмы | 2      | 200               | 400         |

Прогнозная себестоимость десяти образцов: ~17 650 руб.

##### 3.1.2 Экологическая оценка

– Материалы:

Положительные аспекты:

- Корпус из PLA-пластика – биоразлагаем при промышленном компостировании
- Основные компоненты (ESP-12F, MPU6050) – малогабаритные, небольшое количество сырья
- Использование бессвинцового припоя – снижение токсичности

Отрицательные аспекты:

- Li-Po аккумулятор содержит кобальт и литий – добыча ресурсоёмка
- Электронные компоненты содержат следовые количества тяжёлых металлов

- PLA из кукурузы/сахарного тростника – конкуренция с пищевыми культурами

– Технология производства:

Положительные аспекты:

- Локальная 3D-печать – снижение транспортного следа
- Аддитивное производство – минимальные отходы (обрезки PLA идут на переплавку)
- Ручная сборка – низкое энергопотребление на производстве

Отрицательные аспекты:

- Энергозатраты на печать –  $\sim 0,5$  кВт·ч на куб
- Выбросы при пайке – летучие органические соединения от флюса
- Сложность автоматизации – мелкосерийное производство менее эффективно

– Эксплуатация:

Положительные аспекты:

- Сверхнизкое энергопотребление –  $0,02$  кВт·ч/неделю на устройство
- Долговечность – 5+ лет без замены компонентов
- Экономия бумаги – 1 куб заменяет  $\sim 500$  бумажных карточек в год
- Бесшумная работа – отсутствие акустического загрязнения

Отрицательные аспекты:

- Необходимость периодической подзарядки – потребление энергии из сети
- Тепловыделение – незначительный нагрев при передаче данных
- Электромагнитное излучение – низкоуровневое излучение Wi-Fi (в пределах норм)

– Утилизация:

Положительные аспекты:

- Раздельная переработка компонентов:

PLA-корпус → промышленный компостинг

Платы → аффинаж драгметаллов (золото, медь)

Аккумулятор → извлечение лития и кобальта

- Модульность – лёгкость разборки для сортировки
- Маркировка материалов – упрощение сортировки

Отрицательные аспекты:

- Отсутствие инфраструктуры для переработки PLA в России
- Токсичность при сжигании электроники
- Риск попадания аккумуляторов на свалки – загрязнение почвы и воды

### **3.2 Анализ и обобщение результатов проектной работы**

Разработанное проектное изделие — «Интеллектуальный куб» — полностью соответствует требованиям, сформулированным на поисково-исследовательском этапе. Устройство обеспечивает бесшумную тактильную коммуникацию в учебном пространстве: ученик выбирает статус простым поворотом куба, а учитель в реальном времени визуализирует состояние всего класса на экране компьютера. Габариты куба (110×110×110 мм), его автономность (5–7 дней работы от аккумулятора 20 000 мА·ч) и интуитивный физический интерфейс (6 граней — 6 статусов) отвечают заявленным критериям удобства, долговечности и эффективности.

Конструктивно изделие построено по модульному принципу «6+1»: шесть граней и внутренний каркас печатаются отдельно, что упрощает ремонт и замену элементов. В качестве конструкционного материала выбран биоразлагаемый PLA-пластик, за счёт многослойной печати удалось

совместить прочность внешнего корпуса с амортизирующими свойствами внутренних структур.

Таким образом, проект демонстрирует не только техническую реализуемость, но и комплексную инновационность — от применения экологичных материалов и оригинальной технологии производства до разработки специализированного программного обеспечения. Это решение эффективно закрывает проблему оперативной невербальной обратной связи в образовательном процессе и имеет потенциал для масштабирования как в школах, так и в других сферах групповой коммуникации.

### **3.3 Предложения по дальнейшему развитию проекта**

Проект «Интеллектуальные кубы» обладает значительным потенциалом как для массового тиражирования, так и для масштабирования на новые рынки.

Перспектива тиражируемости заключается в переходе от кустарного прототипирования к индустриальным методам. Ключевым шагом станет замена 3D-печати PLA-пластиком на литьё АБС под давлением. Это кардинально снизит себестоимость корпуса и время его изготовления. Для запуска серийного выпуска потребуется разработать пресс-форму и организовать конвейерную линию с автоматической установкой электронных компонентов. Экономика масштаба позволит снизить общую себестоимость готового устройства с 1765 до 1100-1300 рублей при выпуске от 100 штук в месяц. Для этого необходимы инвестиции в оборудование и оснастку, которые окупятся в течение 3-4 месяцев при полной загрузке. Параллельно требуется пройти обязательную сертификацию изделия как учебного оборудования, что откроет путь к поставкам в государственные образовательные учреждения.

Перспектива масштабируемости (выход на новые рынки) основана на универсальности идеи тактильной бесшумной коммуникации. Проект может быть адаптирован для совершенно разных целевых аудиторий. Для корпоративного сектора можно создать версию для совещаний с интеграцией в Zoom или MS Teams, где кубы будут отображать статусы «Согласен», «Вопрос», «Требуется данные». В медицинских учреждениях устройство, размещённое у кровати пациента, позволит ему беззвучно сообщать о базовых потребностях («Боль», «Требуется помощь»), что особенно ценно в отделениях реанимации или для пациентов с нарушениями речи. Для инклюзивного образования можно разработать специализированные кубы с тактильными метками (шрифт Брайля) и упрощёнными пиктограммами для детей с расстройствами аутистического спектра или нарушениями слуха. Наконец, в высшем образовании система может использоваться для мгновенных опросов во время лекций и сбора обратной связи в реальном времени. Каждое из этих направлений потребует незначительной модификации прошивки и дизайна корпуса при сохранении базовой технологической платформы, что делает масштабирование экономически эффективным.

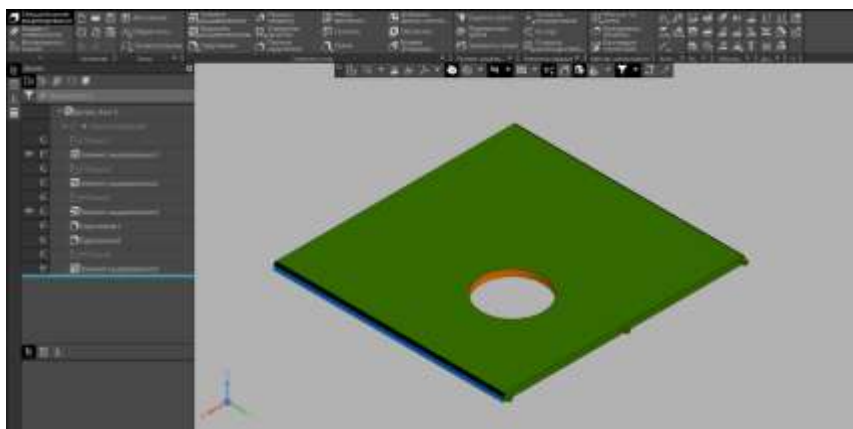
Таким образом, проект готов к эволюции от школьного прототипа до коммерческого продукта с широким спектром применений, что обеспечит его устойчивое развитие и значительный социально-экономический эффект.



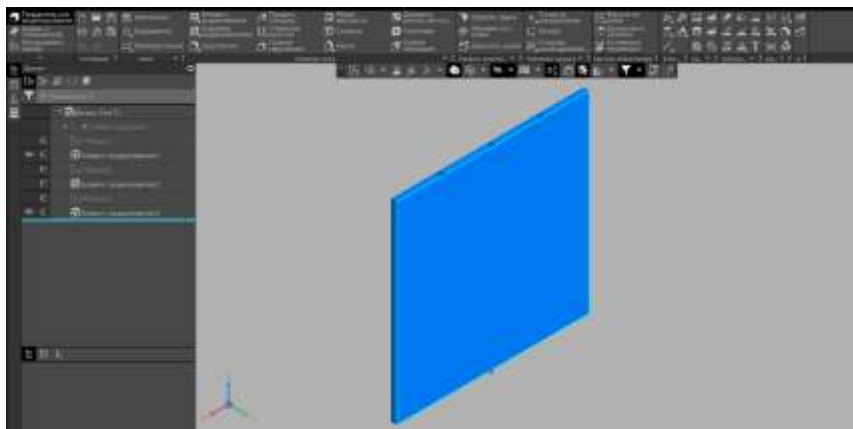
#### 4 СПИСОК ИСТОЧНИКОВ

- 1 Датчик инерционный MPU-6050. Техническое описание: пер. с англ. // InvenSense Inc. – 2013. – 53 с. – Режим доступа: <https://invensense.tdk.com/wp-content/uploads/2015/02/MPU-6000-Datasheet1.pdf>.
- 2 Микроконтроллер ESP8266. Техническое описание: пер. с англ. // Espressif Systems. – 2020. – 90 с. – Режим доступа: [https://www.espressif.com/sites/default/files/documentation/esp8266-technical\\_reference\\_en.pdf](https://www.espressif.com/sites/default/files/documentation/esp8266-technical_reference_en.pdf).
- 3 Светодиодный модуль WS2812B. Техническое описание: пер. с англ. // WorldSemi. – 2020. – 18 с. – Режим доступа: <https://cdn-shop.adafruit.com/datasheets/WS2812B.pdf>.
- 4 Adafruit MPU6050 Library: библиотека для Arduino IDE : офиц. репозиторий на GitHub / Adafruit Industries. – 2023. – Режим доступа: [https://github.com/adafruit/Adafruit\\_MPU6050](https://github.com/adafruit/Adafruit_MPU6050).
- 5 Adafruit NeoPixel Library: библиотека для Arduino IDE : офиц. репозиторий на GitHub / Adafruit Industries. – 2023. – Режим доступа: [https://github.com/adafruit/Adafruit\\_NeoPixel](https://github.com/adafruit/Adafruit_NeoPixel).
- 6 Интернет вещей (IoT) в образовании: систематический обзор / О. Ю. Маркер, К. Л. Смит // Образовательные технологии и общество. – 2022. – Т. 25, № 2. – С. 45–67.
- 7 Стоимость прерванной работы: больше скорости и стресса : исследование Университета Калифорнии в Ирвайне : пер. с англ. / под ред. Г. Марк. – 2018. – Режим доступа: <https://www.ics.uci.edu/~gmark/chi08-mark.pdf>.

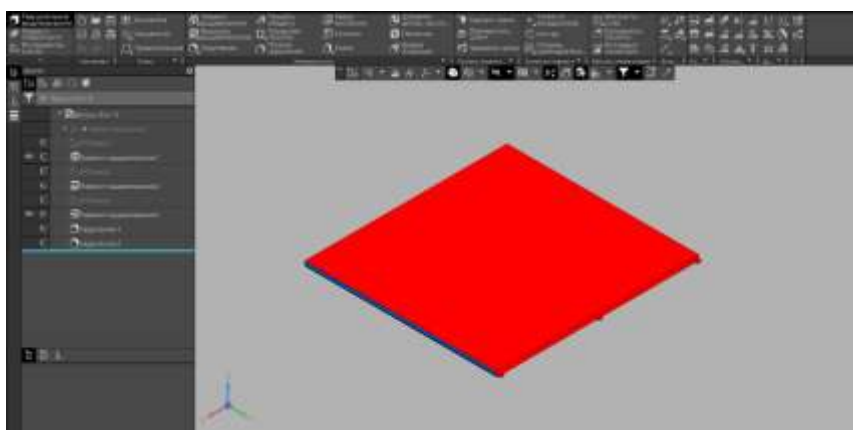
### Фото 3D-моделей



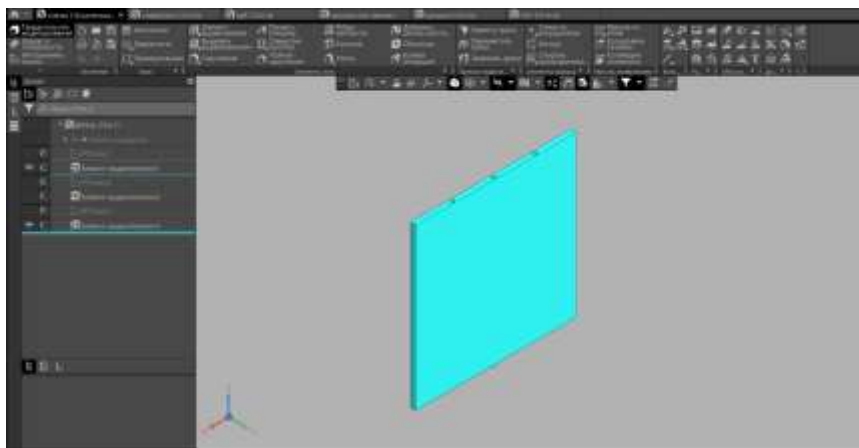
*Рисунок 5 Стена с отверстием для аккумулятора*



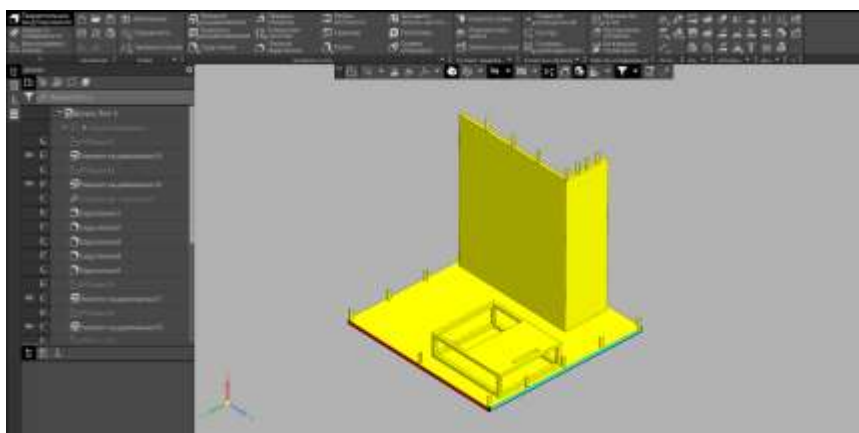
*Рисунок 6 Стена синяя*



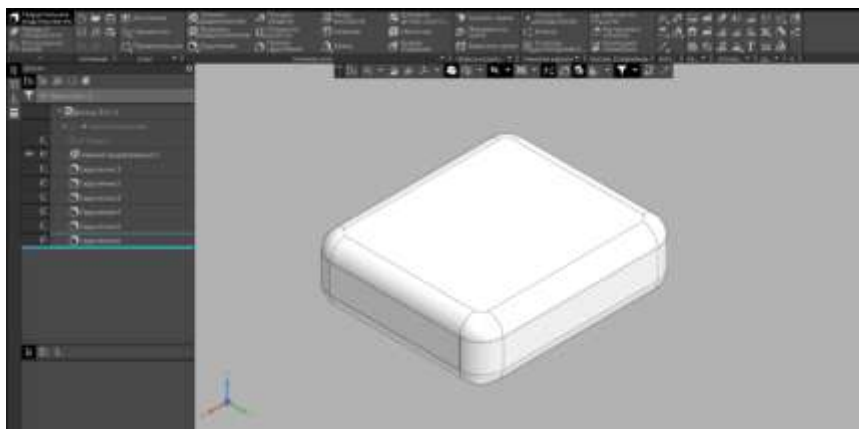
*Рисунок 7 Стена красная*



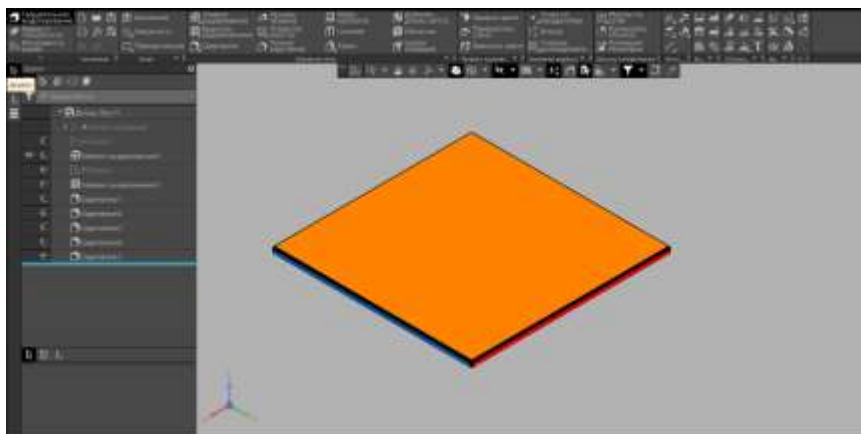
*Рисунок 8 Стена берюзовая*



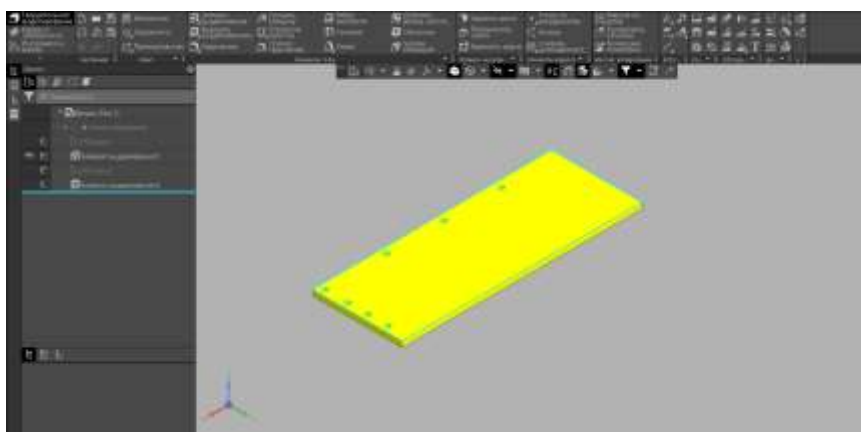
*Рисунок 9 Основа с держателем для ESP и аккумулятора*



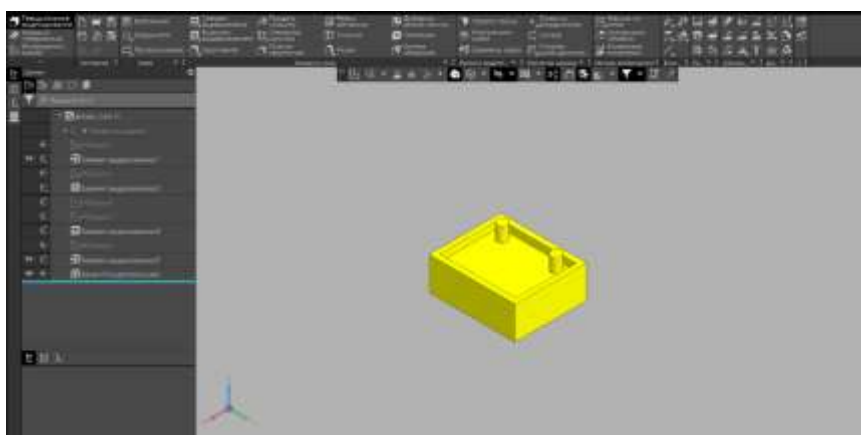
*Рисунок 10 Модель аккумулятора*



*Рисунок 11 Крыша оранжевая*



*Рисунок 12 Крыша для аккумулятора*



*Рисунок 13 Держатель датчика MPU*

### Программная реализация(код)

```
#!/usr/bin/env python3

# esp_squares_monitor.py

import tkinter as tk

from tkinter import ttk, messagebox

import requests

import threading

import time

import json

from datetime import datetime

from collections import Counter

import random # Для эмуляции, если ESP
не доступен

class ESPSquaresMonitor:

    def __init__(self, root):

        self.root = root

        self.root.title("ESP Squares Monitor")

        self.root.geometry("1200x800")

        # Конфигурация

        self.esp_ip = "192.168.137.176" # IP
вашего ESP

        self.update_interval = 5 # секунд

        self.total_squares = 10

        self.is_monitoring = False

        # Цветовая схема для значений 1-6

        self.color_map = {

            1: "#2ecc71", # зеленый

            2: "#e74c3c", # красный

            3: "#3498db", # синий

            4: "#f1c40f", # желтый

            5: "#e67e22", # оранжевый

            6: "#ecf0f1", # белый

            0: "#95a5a6" # серый (по
умолчанию/нет данных)

        } # Состояние квадратов

        self.squares_state = [0] *
self.total_squares # 0 = нет данных

        # Для первого ESP (реальный)

        self.esp_state = 0 # Текущее значение
с ESP

        # Статистика

        self.history = {i: [] for i in
range(self.total_squares)}

        self.total_counts = Counter()

        self.setup_ui()

    def setup_ui(self):

        # Главный контейнер

        main_frame = ttk.Frame(self.root)
```

```

main_frame.pack(fill="both",
expand=True, padx=10, pady=10)

# Верхняя панель - заголовок и
управление

top_frame = ttk.Frame(main_frame)

top_frame.pack(fill="x", pady=(0, 20))

title = ttk.Label(top_frame, text="🚦
ESP Squares Monitor",

font=("Arial", 24, "bold"))

title.pack(side="left", padx=10)

# Панель управления

control_frame = ttk.Frame(top_frame)

control_frame.pack(side="right",
padx=10)

self.ip_var =
tk.StringVar(value=self.esp_ip)

ip_entry = ttk.Entry(control_frame,
textvariable=self.ip_var, width=15)

ip_entry.pack(side="left", padx=2)

self.interval_var =
tk.StringVar(value=str(self.update_interval))

interval_spin =
tk.Spinbox(control_frame, from_=1, to=60,
textvariable=self.interval_var, width=5)

interval_spin.pack(side="left", padx=2)

ttk.Label(control_frame,
text="сек").pack(side="left", padx=2)

```

```

self.start_btn =
tk.Button(control_frame, text="▶ Старт",
command=self.toggle_monitoring)

self.start_btn.pack(side="left", padx=2)

tk.Button(control_frame, text="🔄
Обновить",

command=self.manual_update).pack(side="l
eft", padx=2)

tk.Button(control_frame, text="📊
Статистика",

command=self.show_stats).pack(side="left",
padx=2)

tk.Button(control_frame, text="⚙️
Настройки",

command=self.show_settings).pack(side="le
ft", padx=2)

# Основная область - квадраты

squares_frame =
tk.LabelFrame(main_frame, text="ESP
Squares (10 устройств)")

squares_frame.pack(fill="both",
expand=True, pady=(0, 20))

# Сетка 2x5 для квадратов

grid_frame = ttk.Frame(squares_frame)

grid_frame.pack(expand=True,
padx=20, pady=20)

self.square_canvases = []

```

```

self.square_labels = []

for i in range(self.total_squares):

    # Фрейм для каждого квадрата

    square_frame =
    ttk.Frame(grid_frame, relief="ridge",
borderwidth=2)

    row = i // 5

    col = i % 5

    square_frame.grid(row=row,
column=col, padx=10, pady=10,
sticky="nsew")

    # Заголовок квадрата

    title = ttk.Label(square_frame,
text=f"ESP #{i+1}",

font=("Arial", 12, "bold"))

    title.pack(pady=(5, 0))

    # Canvas для цветного квадрата

    canvas = tk.Canvas(square_frame,
width=100, height=100,

bg=self.color_map[0],
highlightthickness=0)

    canvas.pack(pady=5)

    # Рисуем квадрат

    canvas.create_rectangle(10, 10, 90,
90, fill=self.color_map[0],

outline="black",
width=2)

    # Метка с текущим значением

```

```

value_label =
    ttk.Label(square_frame, text="--",

font=("Arial", 16,
"bold"))

    value_label.pack(pady=(0, 5))

    # Статус подключения

    status_label =

    ttk.Label(square_frame, text="✗ Нет
данных",

font=("Arial", 8),
foreground="gray")

    status_label.pack(pady=(0, 5))

    self.square_canvases.append({

        "canvas": canvas,

        "square": canvas.find_all()[0], #
ID квадрата

        "status": status_label

    })

    self.square_labels.append(value_label)

    # Делаем все колонки одинаковой
ширины

    grid_frame.columnconfigure(col,
weight=1)

    # Нижняя панель - статистика по
цветам

    stats_frame =

    ttk.LabelFrame(main_frame,
text="Статистика по цветам")

```

```

stats_frame.pack(fill="x", pady=(0, 10))

# Фрейм для цветовых индикаторов
colors_frame = ttk.Frame(stats_frame)

colors_frame.pack(pady=10)

self.color_stats_labels = {}

for value, color in
self.color_map.items():

    if value == 0:

        continue # Пропускаем серый
ЦВЕТ

        color_frame =
        ttk.Frame(colors_frame)

        color_frame.pack(side="left",
        padx=15)

        # Цветной квадратик

        color_canvas =
        tk.Canvas(color_frame, width=30,
        height=30,

        bg=color,
        highlightthickness=1)

        color_canvas.pack()

        color_canvas.create_rectangle(2, 2,
        28, 28, fill=color,

        outline="black")

        # Описание и счетчик

        color_names = {

            1: "Зеленый", 2: "Красный", 3:
"Синий",

```

```

4: "Желтый", 5: "Оранжевый", 6:
"Белый"

        }

        ttk.Label(color_frame,
        text=color_names[value]).pack()

        count_label = ttk.Label(color_frame,
        text="0",

        font=("Arial", 14,
        "bold"))

        count_label.pack()

        self.color_stats_labels[value] =
        count_label

        # Общая статистика

        self.total_label = ttk.Label(stats_frame,

        text="Всего
обновлений: 0",

        font=("Arial", 10))

        self.total_label.pack(pady=5)

        # Статус бар

        self.status_var =
        tk.StringVar(value="  Готов к работе")

        status_bar = ttk.Label(self.root,
        textvariable=self.status_var,

        relief="sunken",
        anchor="w")

        status_bar.pack(side="bottom",
        fill="x")

```



```

# Легенда

legend_frame = ttk.Frame(main_frame)

legend_frame.pack(fill="x", pady=(10,
0))

ttk.Label(legend_frame,
text="Легенда:", font=("Arial", 10,
"bold")).pack(side="left", padx=5)

for value in range(1, 7):

    color_names = ["Зеленый",
"Красный", "Синий", "Желтый",
"Оранжевый", "Белый"]

    legend_item =
ttk.Frame(legend_frame)

    legend_item.pack(side="left",
padx=5)

    tk.Canvas(legend_item, width=15,
height=15,

bg=self.color_map[value]).pack(side="left")

    ttk.Label(legend_item,
text=f"={value} ({color_names[value-
1]})").pack(side="left")

# Для первого ESP (реальный)
показываем его IP

esp_info = ttk.Label(main_frame,

text=f"ESP #1
подключается к: {self.esp_ip}",

font=("Arial", 10, "italic"))

esp_info.pack(pady=5)

```

```

# Эмуляция данных для остальных
ESP

self.emulate_check =
tk.BooleanVar(value=True)

emulate_checkbox =
ttk.Checkbutton(main_frame,

text="Эмулировать данные для
ESP #2-10",

variable=self.emulate_check)

emulate_checkbox.pack()

def get_esp_data(self):

    """Получение данных с реального
ESP"""

    try:

        response =
requests.get(f"http://{self.esp_ip}/random",
timeout=2)

        if response.status_code == 200:

            value = int(response.text.strip())

            if 1 <= value <= 6:

                return value

        except

requests.exceptions.RequestException as e:

            print(f"Ошибка подключения к
ESP: {e}")

            return None # Если не удалось
получить данные

def emulate_esp_data(self, esp_num):

```

```

        """Эмуляция данных для ESP (кроме
        первого)"""

        if esp_num == 0: # Первый ESP -
        реальный

            return self.esp_state if self.esp_state
            != 0 else None

        # Для остальных ESP эмулируем
        данные

        if self.emulate_check.get():

            # Эмуляция: иногда возвращаем
            случайное значение, иногда ошибку

            if random.random() > 0.1: # 90%
            успешных запросов

                # С небольшим шансом меняем
                значение

                if random.random() > 0.7:

                    return random.randint(1, 6)

                else:

                    # Сохраняем предыдущее
                    значение или новое

                    current =
                    self.squares_state[esp_num]

                    return current if current != 0 else
                    random.randint(1, 6)

            return None

        def update_square(self, esp_num, value):

            """Обновление отображения
            квадрата"""

```

```

        if value is None:

            # Нет данных

            color = self.color_map[0]

            text = "--"

            status = "✗ Нет данных"

            self.squares_state[esp_num] = 0

        else:

            # Есть данные

            color = self.color_map[value]

            text = str(value)

            status = "✓ Данные получены"

            self.squares_state[esp_num] = value

            # Сохраняем в историю

            self.history[esp_num].append({

                "time":
                datetime.now().strftime("%H:%M:%S"),

                "value": value

            })

            # Обновляем Canvas

            canvas_data =
            self.square_canvases[esp_num]

            canvas_data["canvas"].itemconfig(canvas_d
            ata["square"], fill=color)

            canvas_data["status"].config(text=status)

            # Обновляем метку

```

```

self.square_labels[esp_num].config(text=text)
t)

def update_statistics(self):

    """Обновление статистики"""

    # Считаем количество каждого цвета

    counts = Counter(self.squares_state)

    # Обновляем счетчики

    for value in range(1, 7):

        count = counts.get(value, 0)

    self.color_stats_labels[value].config(text=str
(count))

    # Общее количество обновлений

    total_updates = sum(len(h) for h in
self.history.values())

    self.total_label.config(text=f"Всего
обновлений: {total_updates}")

    # Обновляем общий счетчик

    self.total_counts.update(self.squares_state)

    def update_all_squares(self):

        """Обновление всех квадратов"""

        # Обновляем реальный ESP

        esp_value = self.get_esp_data()

        self.esp_state = esp_value if esp_value
else 0

        self.update_square(0, esp_value)

        # Обновляем остальные ESP
(эмулированные)

```

```

for i in range(1, self.total_squares):

    value = self.emulate_esp_data(i)

    self.update_square(i, value)

# Обновляем статистику

self.update_statistics()

# Обновляем статус

success_count = sum(1 for v in
self.squares_state if v != 0)

self.status_var.set(f"✅ Обновлено:
{success_count}/{self.total_squares} |
Следующее обновление через
{self.update_interval} сек")

def manual_update(self):

    """Ручное обновление"""

    if not self.is_monitoring:

        self.update_all_squares()

def monitoring_loop(self):

    """Цикл мониторинга"""

    while self.is_monitoring:

        try:

            self.update_all_squares()

        except Exception as e:

            self.status_var.set(f"❌ Ошибка:
{str(e)[:50]}")

            # Ждем указанный интервал

            for i in range(self.update_interval *
10): # Проверяем каждые 0.1 сек

```

```

        if not self.is_monitoring:

            return

            time.sleep(0.1)

    def toggle_monitoring(self):

        """Включение/выключение
        мониторинга"""

        if not self.is_monitoring:

            # Начинаем мониторинг

            self.is_monitoring = True

            self.start_btn.config(text="⏸
            Стоп")

            # Обновляем конфигурацию

            try:

                self.esp_ip = self.ip_var.get()

                self.update_interval =
                int(self.interval_var.get())

            except:

                messagebox.showerror("Ошибка",
                "Некорректные настройки")

                self.is_monitoring = False

                self.start_btn.config(text="▶
                Старт")

            return

            # Запускаем поток мониторинга

            self.monitor_thread =
            threading.Thread(target=self.monitoring_loo
            p, daemon=True)

```

```

        self.monitor_thread.start()

        self.status_var.set("🚀 Мониторинг
        запущен")

        else:

            # Останавливаем мониторинг

            self.is_monitoring = False

            self.start_btn.config(text="▶
            Старт")

            self.status_var.set("⏸ Мониторинг
            остановлен")

            def show_stats(self):

                """Показать подробную
                статистику"""

                # Создаем новое окно

                stats_window = tk.Toplevel(self.root)

                stats_window.title("Подробная
                статистика")

                stats_window.geometry("800x600")

                # Создаем Notebook для вкладок

                notebook =
                ttk.Notebook(stats_window)

                notebook.pack(fill="both",
                expand=True, padx=10, pady=10)

                # Вкладка 1: Общая статистика

                general_frame = tk.Frame(notebook)

                notebook.add(general_frame, text="📊
                Общая статистика")

```

```

# Текстовое поле для статистики

stats_text = tk.Text(general_frame,
wrap="word", font=("Consolas", 10))

scrollbar = ttk.Scrollbar(general_frame,
orient="vertical",
command=stats_text.yview)

stats_text.configure(yscrollcommand=scroll
bar.set)

scrollbar.pack(side="right", fill="y")

stats_text.pack(side="left", fill="both",
expand=True)

# Генерируем статистику

stats_lines = []

stats_lines.append("=" * 60)

stats_lines.append("ОБЩАЯ
СТАТИСТИКА ESP КВАДРАТОВ")

stats_lines.append("=" * 60)

stats_lines.append("")

# Текущее состояние

stats_lines.append("Текущее
состояние квадратов:")

for i in range(self.total_squares):

    value = self.squares_state[i]

    color_name = {

        0: "Серый (нет данных)", 1:
"Зеленый", 2: "Красный",

        3: "Синий", 4: "Желтый", 5:
"Оранжевый", 6: "Белый"

```

```

}.get(value, "Неизвестно")

stats_lines.append(f"    ESP #{i+1}:
значение={value} ({color_name})")

stats_lines.append("")

# Общее распределение цветов

stats_lines.append("Распределение
цветов:")

total_non_zero = sum(1 for v in
self.squares_state if v != 0)

stats_lines.append(f"        Активных
квадратов:
{total_non_zero}/{self.total_squares}")

for value in range(1, 7):

    count = sum(1 for v in
self.squares_state if v == value)

    if count > 0:

        color_name = ["Зеленый",
"Красный", "Синий", "Желтый",
"Оранжевый", "Белый"][value-1]

        percentage = (count /
total_non_zero * 100) if total_non_zero > 0
else 0

        stats_lines.append(f"
{color_name}: {count}
({percentage:.1f}%)")

stats_lines.append("")

# История изменений

stats_lines.append("История
изменений (последние 20):")

```

```

all_events = []

for i in range(self.total_squares):

    for event in self.history[i][-5:]: #
        Последние 5 событий каждого ESP

        all_events.append((event["time"],
        i, event["value"]))

    # Сортируем по времени

    all_events.sort(key=lambda x: x[0],
reverse=True)

    for time_str, esp_num, value in
all_events[:20]:

        color_name = ["Зеленый",
"Красный", "Синий", "Желтый",
"Оранжевый", "Белый"][value-1]

        stats_lines.append(f" [{time_str}]
ESP #{esp_num+1} → {value}
({color_name})")

    # Вставляем текст

    stats_text.insert("1.0",
"\n".join(stats_lines))

    stats_text.config(state="disabled")

    # Вкладка 2: График истории
    (простой текстовый)

    history_frame = ttk.Frame(notebook)

    notebook.add(history_frame, text="📈
История значений")

    # Создаем Canvas для простого
графика

```

```

canvas = tk.Canvas(history_frame,
bg="white")

canvas.pack(fill="both", expand=True,
padx=10, pady=10)

# Рисуем простой график для первого
ESP

if self.history[0]:

    values = [entry["value"] for entry in
self.history[0]]

    times = list(range(len(values)))

    if len(values) > 1:

        # Вычисляем масштаб

        width = 700

        height = 400

        max_val = max(values)

        min_val = min(values)

        # Рисуем оси

        canvas.create_line(50, 50, 50,
height - 50, width=2)

        canvas.create_line(50, height - 50,
width - 50, height - 50, width=2)

        # Рисуем график

        points = []

        for i, val in enumerate(values[-
50:]): # Последние 50 точек

            x = 50 + (i * (width - 100) /
min(49, len(values)-1))

```

```

        y = height - 50 - ((val - min_val)
* (height - 100) / max(1, max_val - min_val))

        points.append((x, y))

        canvas.create_oval(x-3, y-3,
x+3, y+3, fill=self.color_map[val])

    # Соединяем точки

    for i in range(len(points)-1):

        canvas.create_line(points[i][0],
points[i][1],

        points[i+1][0],
points[i+1][1],

        width=2, fill="blue")

    # Подписи

    canvas.create_text(width // 2,
height - 20, text="Время (последние
значения)", font=("Arial", 10))

    canvas.create_text(20, height // 2,
text="Значение", angle=90, font=("Arial",
10))

    # Легенда значений

    legend_y = 20

    for val in range(1, 7):

        canvas.create_rectangle(60,
legend_y, 80, legend_y + 15,

fill=self.color_map[val], outline="black")

        color_name = ["Зеленый",
"Красный", "Синий", "Желтый",
"Оранжевый", "Белый"][val-1]

```

```

        canvas.create_text(100,
legend_y + 7, text=f"= {val}
({color_name})",

        anchor="w",
font=("Arial", 9))

        legend_y += 25

    # Кнопка экспорта

    export_btn = ttk.Button(stats_window,
text="📄 Экспорт данных",
command=self.export_data)

    export_btn.pack(pady=10)

    def show_settings(self):

        """"Окно настроек""""

        settings_window =
tk.Toplevel(self.root)

        settings_window.title("Настройки")

        settings_window.geometry("400x300")

        ttk.Label(settings_window,
text="Настройки ESP Squares Monitor",

        font=("Arial", 14,
"bold")).pack(pady=10)

    # Настройки соединения

    conn_frame =
ttk.LabelFrame(settings_window,
text="Настройки соединения")

    conn_frame.pack(fill="x", padx=20,
pady=10)

```

```

        ttk.Label(conn_frame, text="IP адрес
ESP:").grid(row=0, column=0, padx=5,
pady=5, sticky="w")

```

```

        ip_entry = ttk.Entry(conn_frame,
textvariable=self.ip_var, width=20)

```

```

        ip_entry.grid(row=0, column=1,
padx=5, pady=5)

```

```

        ttk.Label(conn_frame, text="Интервал
обновления (сек):").grid(row=1, column=0,
padx=5, pady=5, sticky="w")

```

```

        interval_entry = ttk.Entry(conn_frame,
textvariable=self.interval_var, width=10)

```

```

        interval_entry.grid(row=1, column=1,
padx=5, pady=5, sticky="w")

```

# Настройки эмуляции

```

        emul_frame =
ttk.LabelFrame(settings_window,
text="Настройки эмуляции")

```

```

        emul_frame.pack(fill="x", padx=20,
pady=10)

```

```

        emulate_var =
tk.BooleanVar(value=self.emulate_check.get
())

```

```

        emulate_check =
ttk.Checkbutton(emul_frame,
text="Эмулировать данные для ESP #2-
10",
variable=emulate_var)

```

```

        emulate_check.grid(row=0, column=0,
columnspan=2, padx=5, pady=5,
sticky="w")

```

```

def main():

```

```

    root = tk.Tk()

```

```

    app = ESPSquaresMonitor(root)

```

```

    # Запускаем цикл обработки событий

```

```

    root.mainloop()

```

```

if __name__ == "__main__":

```

```

    main()

```